

MID-TERM REPORT

Using Deep Reinforcement Learning to Optimise a Stock-Trading Strategy and Maximise Investment Return

Algorithm studied: **Deep Q-Network (DQN)** | Benchmark solved: **Inverted Pendulum (CartPole-v1)**

Group members: [Member 1], [Member 2], [Member 3]

1. Introduction and Motivation

Our project applies deep reinforcement learning (RL) to stock trading: an agent observes market information and decides when to buy, sell, or hold in order to maximise long-term investment return. Trading maps naturally onto the RL formalism — the *state* is market data (prices, technical indicators) plus the current portfolio; the *actions* are buy/sell/hold; and the *reward* is the change in portfolio value. Because the reward for a decision may arrive many steps later, supervised learning is a poor fit, whereas RL is designed precisely for such delayed-reward sequential decision problems.

For this mid-term report we (i) explain the Deep Q-Network (DQN) algorithm — its theory, core concepts, advantages, and limitations — and (ii) validate our own from-scratch DQN implementation on the classic **inverted pendulum** control problem (CartPole-v1), as required by the task statement (DQN instead of DDPG). The pendulum experiment exercises every component we will reuse for trading in the final report.

2. Background: Reinforcement Learning and Q-Learning

An RL problem is a Markov Decision Process (MDP) (S, A, P, R, γ) : at each step the agent observes state s , chooses action a , receives reward r , and transitions to s' . The goal is a policy $\pi(a|s)$ maximising the expected discounted return $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$, where the discount factor $\gamma \in [0,1)$ trades off immediate versus future reward.

Q-learning (Watkins, 1989) learns the optimal action-value function $Q^*(s,a)$ — the best achievable return after taking action a in state s . It satisfies the **Bellman optimality equation**:

$$Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s',a')]]$$

Classic Q-learning stores Q in a lookup table and updates it toward the bracketed target. It provably converges for finite MDPs, but a table is impossible when the state is continuous (pendulum angles, stock prices) — the state space must be *generalised over*, not enumerated. Q-learning is **off-policy**: it learns about the greedy policy while behaving with an exploratory one, which is what later lets DQN learn from stored past experience.

3. The Deep Q-Network (DQN) Algorithm

DQN (Mnih et al., *Nature* 2015) replaces the Q -table with a deep neural network $Q(s,a;\theta)$ and famously reached human-level play on Atari games from raw pixels. The network takes the state as input and outputs one Q -value per action, so a full greedy decision costs a single forward pass. Training minimises the squared **temporal-difference (TD) error** between the network's prediction and the Bellman target:

$$L(\theta) = E[(r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

Naively bootstrapping a network toward its own predictions diverges, because (a) consecutive samples are highly correlated, violating the i.i.d. assumption of stochastic gradient descent, and (b) the regression target moves every time the weights move. DQN's two key innovations address exactly these problems.

3.1 Core concept: Experience Replay

Every transition (s, a, r, s', done) is stored in a large circular **replay buffer**. Learning steps sample a random minibatch from the buffer instead of using the latest transition. This (i) breaks temporal correlation between training samples, (ii) reuses each experience many times, improving sample efficiency, and (iii) smooths the data distribution over many past behaviours. It is only valid because Q-learning is off-policy.

3.2 Core concept: Target Network

A second network with parameters θ^- — a periodically synced copy of the online network — computes the Bellman target. Freezing the target between syncs turns each learning phase into a well-posed regression toward a *stationary* target, dramatically stabilising training. We hard-copy the online weights into the target network every 250 gradient steps.

3.3 Core concept: Epsilon-Greedy Exploration

With probability ϵ the agent acts randomly (explore); otherwise it takes $\arg\max_a Q(s, a)$ (exploit). ϵ is annealed from 1.0 to 0.01 (multiplied by 0.995 per episode), so the agent explores broadly at first and increasingly trusts its learned values later.

3.4 Core concept: Huber (smooth-L1) Loss

Squared error explodes for large TD errors early in training; the Huber loss is quadratic near zero but linear beyond a threshold, which is equivalent to clipping the TD-error gradient to $[-1, 1]$. This prevents destabilising parameter jumps.

3.5 Advantages of DQN

DQN **scales to enormous or continuous state spaces** where tabular methods are impossible, and the network generalises between similar states — essential both for the pendulum's continuous state and for market data. It is **sample-efficient** relative to on-policy methods because the replay buffer reuses every transition many times, and being **off-policy** it can in principle learn from logged or historical data — highly relevant to trading, where we train on historical prices. It is also simple to implement and act with: one forward pass yields all action values.

3.6 Limitations of DQN

Discrete actions only. The max over actions is only tractable for a finite action set; continuous control needs actor-critic methods such as DDPG. (For trading we therefore discretise to buy/sell/hold.) **Overestimation bias:** the max operator systematically overestimates Q-values under noise; Double DQN mitigates this. **Instability and hyperparameter sensitivity:** the combination of function approximation, bootstrapping and off-policy learning (the 'deadly triad') means training can diverge or collapse — we observed this directly (Section 5.3). Uniform replay also treats all transitions as equally useful (prioritised replay improves this), and DQN learns only deterministic greedy policies.

4. Solving the Inverted Pendulum with DQN

4.1 The environment

We use Gymnasium's **CartPole-v1**, the standard inverted-pendulum benchmark: a pole is hinged on a cart that slides on a frictionless track, and the agent applies a force LEFT or RIGHT each timestep to keep the pole upright. The state is four continuous values — cart position x , cart velocity, pole angle θ , pole angular velocity. The reward is +1 for every step the pole stays up; an episode ends when $|\theta| > 12^\circ$, $|x| > 2.4$, or after 500 steps. The task is **solved** when the average reward over 100 consecutive episodes reaches **475**. Note the action space is discrete (left/right), which is exactly why DQN — rather than DDPG — is the appropriate algorithm.

4.2 Implementation — from scratch in NumPy

We implemented every component ourselves in NumPy (file **dqn_cartpole.py**): the Q-network with hand-coded backpropagation, the Adam optimiser, the replay buffer, and the training loop — no deep-learning framework — so every moving part of DQN is explicit and inspectable. The Q-network is an MLP $4 \rightarrow 128 \rightarrow 128 \rightarrow 2$ with ReLU activations and He initialisation:

```
class QNetwork:                                # MLP 4 -> 128 -> 128 -> 2
    def forward(self, x):
        self.z1 = x @ W1 + b1; self.a1 = np.maximum(self.z1, 0) # ReLU
        self.z2 = self.a1 @ W2 + b2; self.a2 = np.maximum(self.z2, 0)
        return self.a2 @ W3 + b3 # one Q-value per action

    def backward(self, dQ):                     # hand-coded backprop
        gw3 = self.a2.T @ dQ; da2 = dQ @ W3.T; dz2 = da2 * (self.z2 > 0)
        gw2 = self.a1.T @ dz2; da1 = dz2 @ W2.T; dz1 = da1 * (self.z1 > 0)
        gw1 = self.x.T @ dz1
        return gradients                        # then one Adam step
```

Action selection is ϵ -greedy, and the learning step implements the TD update with the target network and Huber gradient clipping:

```
def learn(self):                               # one gradient step
    s, a, r, s2, d = self.buffer.sample(BATCH_SIZE) # random minibatch (64)

    q_next = self.target.forward(s2).max(1)        # frozen target net
    y = r + GAMMA * q_next * (1.0 - d)             # Bellman target

    q_all = self.policy.forward(s)                 # online net Q(s,.)
    q_sa = q_all[np.arange(BATCH_SIZE), a]         # Q(s,a) actually taken

    td = q_sa - y
    dq_sa = np.clip(td, -1.0, 1.0) / BATCH_SIZE   # Huber gradient
    dQ = zeros_like(q_all); dQ[range(B), a] = dq_sa
    self.policy.adam_step(self.policy.backward(dQ)) # backprop + Adam

    if self.learn_steps % TARGET_UPDATE == 0:      # every 250 steps
        self.target.copy_from(self.policy)         # sync target net
```

How it works, step by step. Each environment step the agent (1) picks an action ϵ -greedily from the online network, (2) stores the transition $(s, a, r, s', \text{terminated})$ in the 50,000-slot replay buffer, and (3) performs one learning step: sample 64 random past transitions, compute the Bellman target $y = r + \gamma \max_a Q_{\text{target}}(s', a')$ (zeroing the bootstrap term for true terminal states), take the gradient of the Huber loss between $Q(s, a)$ and y — non-zero only at the action actually taken — backpropagate it, and apply one Adam update. A subtlety: episodes cut off by the 500-step time limit are *truncated*, not terminal, so we still bootstrap from their successor state; treating them as terminal would wrongly teach the agent that surviving 500 steps yields no future value. After each episode ϵ decays, and whenever the greedy policy achieves a new best evaluation score its weights are snapshotted to **best_model.npz**.

4.3 Hyperparameters

Parameter	Value	Parameter	Value
discount factor γ	0.99	replay buffer size	50,000
learning rate (Adam)	5e-4	learn start (min buffer)	1,000
batch size	64	target-net sync	every 250 steps
ϵ start / end / decay	1.0 / 0.01 / 0.995	network	4-128-128-2, ReLU

5. Results and Findings

5.1 Learning curve

Training solved CartPole-v1 in **785 episodes**: the 100-episode moving average crossed the 475 threshold (reaching 476.6). Early episodes last only 10–30 steps while the agent explores almost randomly; once the replay buffer fills and ϵ decays, performance rises steeply, with many episodes hitting the 500-step cap.

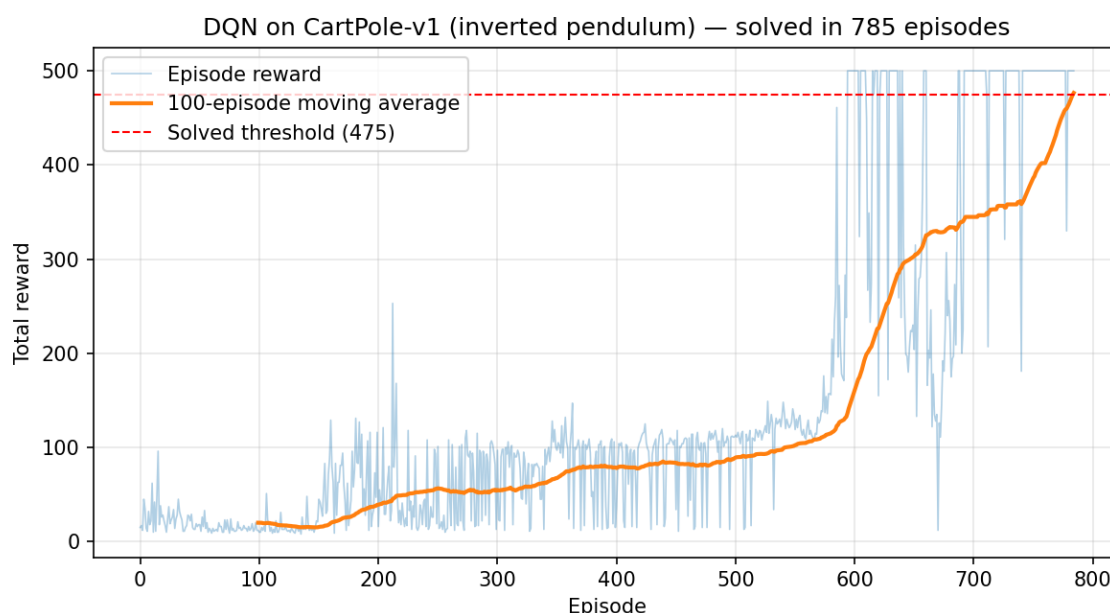


Figure 1: Per-episode reward (blue), 100-episode moving average (orange) and the solved threshold of 475 (dashed red). Solved at episode 785.

5.2 Final performance

The best-snapshot agent, evaluated *greedily* ($\epsilon = 0$) on **20 completely unseen starting states**, balanced the pole for the full 500 steps in **every single episode** — a **perfect mean score of 500/500**. Qualitatively, an untrained agent drops the pole within about a second; the trained agent balances indefinitely.

5.3 Empirical confirmation of DQN's limitations

Two documented weaknesses of DQN appeared directly in our experiments. (1) **Hyperparameter sensitivity**: our first run (learning rate 1e-3, ϵ -decay 0.99, target sync every 500 steps) plateaued around reward 100–160 and never solved the task; reducing the learning rate to 5e-4, slowing exploration decay to 0.995 and syncing the target every 250 steps solved it. (2) **Catastrophic**

forgetting: simply continuing training *past* the solved point made performance collapse from ~500 to ~10 steps per episode — the greedy data distribution narrows, the network overfits it, and earlier competence is destroyed. Best-model snapshotting (keeping the weights with the best greedy evaluation) is our practical remedy, and it is exactly the model-selection discipline we will need when training the trading agent.

6. From the Pendulum to Stock Trading (Next Steps)

The pendulum validated our full DQN pipeline end-to-end. For the final report we transfer it to trading by changing only the environment: the **state** becomes a window of price history (OHLCV), technical indicators, cash balance and current holdings; the **actions** become the discrete set {buy, sell, hold}; and the **reward** becomes the step change in portfolio value (with transaction costs). Because DQN is off-policy, we can train on historical market data via the replay buffer. Planned refinements motivated by Section 5.3: Double DQN against overestimation, and walk-forward evaluation with best-model snapshotting to avoid overfitting to a single market regime.

7. Group Member Contributions

[Member 1] — Theory and literature review: Q-learning and DQN theory (Sections 2–3), analysis of advantages/limitations, report writing. **[Member 2] — Implementation:** from-scratch NumPy Q-network with backprop and Adam, replay buffer, agent and training loop (`dqn_cartpole.py`), code documentation. **[Member 3] — Experiments and evaluation:** training runs, hyperparameter tuning, greedy evaluation protocol, learning-curve analysis and figures, findings in Section 5. All members contributed to the accompanying video.

References

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature* 518, 529–533, 2015. [2] V. Mnih et al., “Playing Atari with Deep Reinforcement Learning,” arXiv:1312.5602, 2013. [3] C. Watkins and P. Dayan, “Q-learning,” *Machine Learning* 8, 279–292, 1992. [4] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. [5] H. van Hasselt et al., “Deep Reinforcement Learning with Double Q-learning,” AAAI 2016. [6] Gymnasium documentation, CartPole-v1, Farama Foundation, 2024.